



BIBLIOTECA

Alumnos: Julieta Rodríguez, Francisco Demartino

Comisión: 1

Universidad: Universidad Nacional de Quilmes

Espacio curricular: Programación con Objetos 1

Cuatrimestre y Año: Segundo cuatrimestre, 2025

Universidad nacional de Quilmes
Departamento de Ciencia y Tecnología
Programación con Objetos 1

Fecha de entrega: 4/12/2025

Alcance

Se implementó la totalidad de los requerimientos solicitados en el enunciado asignado (biblioteca), incluyendo el registro de socios y libros, la gestión de ejemplares, préstamos y devoluciones, y la aplicación automática de sanciones.

Se respetaron las consignas, respecto a límites de préstamos por socio (máximo 5), restricciones de horario (días hábiles) y validaciones de membresía. Se agregaron ciertos casos borde que contemplamos necesarios.

Además, se implementó el Bonus de búsqueda de libros, permitiendo realizar búsquedas complejas por autor, fecha, temas o cualquier combinación de estos.

Modelo

Los objetos principales del sistema y sus responsabilidades son:

- **Biblioteca:** Es responsable de validar las reglas institucionales (horarios, disponibilidad en catálogo) y coordinar la interacción entre el Socio y los libros.
- **Socio:** Representa al usuario del sistema, conoce su historial, sus préstamos en curso y su cantidad de sanciones. Es responsable de validar si está apto para pedir un nuevo préstamo (no estar vetado, cuota al día).
- **Libro:** Su responsabilidad principal es administrar el stock de sus Ejemplares físicos. Además, almacena sus datos (ISBN, autor, título, fecha de publicación, idioma y lista de temas.).

- Ejemplar: Representa una unidad de un libro. Almacena su número de inventario, su fecha de ingreso y el libro al que pertenece.
- Préstamo: Es la relación que se establece entre un Socio y un Ejemplar durante un periodo de tiempo. Es responsable de conocer su fecha de vencimiento y delegar la política de renovación a su tipo de préstamo.

Progreso

El trabajo se construyó de manera iterativa e incremental guiada por pruebas (TDD).

Inicialmente, se modeló una relación simple donde el Socio tenía una colección de Libros. Al incorporar la lógica de vencimientos, se detectó la necesidad de agregar el concepto de Préstamo para guardar fechas específicas sin duplicar datos en el libro.

Posteriormente, se refactorizó el sistema para distinguir entre Libro y Ejemplar físico, modificando la clase Libro para que actúe como administradora de stock.

Finalmente, a partir de la base implementada se fueron desarrollando las consignas paso a paso.

Dificultades

Al comienzo del trabajo nos surgió la dificultad de la interpretación del dominio, al no entender la diferencia entre libro y ejemplares. Este problema hizo que comencemos realizando un trabajo y casi al final lo cambiemos por completo.

Las principales problemáticas fueron la utilización de fechas, más que nada la decisión de que clase de fecha utilizar, ya que cada clase tenía sus

mensajes, de los cuales algunos eran muy necesarios para la implementación del trabajo. Terminamos por utilizar `FixedGregorianCalendar`, el cual abarcaba todos los requerimientos que buscábamos.

Con respecto a los préstamos, nuestra mayor dificultad fue a la hora de la aplicación de sanciones, donde tuvimos que realizar una clase llamada “`EstadoDeDevolucion`” que se encargue de elegir la sanción correspondiente en cada caso.

Finalmente, al momento de realizar el bonus nos encontramos con otro problema de comprensión del enunciado al momento de aplicar filtro con fecha de inicio y fin, ya que no sabíamos a que fecha hacia referencia. Además de la implementación del bonus que fue un real desafío.

Facilidades

Nuestras principales facilidades surgieron de los requerimientos de cada acción, es decir los self require. Pudimos realizarlos todos e incluso agregar algunos que no se especificaron en el enunciado concretamente. Reconocer el tipo de lista e implementarla también fue una gran facilidad, al igual que la implementación de los mensajes categorizados como “initialization”, “testing” y “accessing”.

Conclusiones

Gracias a este proyecto pudimos poner en práctica todos los conocimientos adquiridos durante la cursada, tales como el polimorfismo, Test-Driven Development y el encapsulamiento.

La técnica de diseño TDD es muy útil, ya que tiene un enfoque en las necesidades de la implementación, por lo tanto, es menos probable escribir mensajes innecesarios.

El polimorfismo, por otro lado, permite que objetos de diferentes tipos sean tratados como si fueran de un tipo común, y que un mismo método pueda tener comportamientos diferentes según la subclase que lo implemente.